

Prague University of Economics and Business

Faculty of Informatics and Statistics



Graph Neural Networks for Image Classification: An Empirical Comparison with Convolutional Networks

BACHELOR THESIS

Study program: Data Analytics

Author: Hedvika Rychnovská

Bachelor Thesis Supervisor: doc. Ing. Ondřej Zamazal, Ph.D.

Prague, December 2025

Acknowledgement

I would like to sincerely thank my thesis supervisor doc. Ing. Ondřej Zamazal, Ph.D. for his speedy and insightful feedback, valuable advice, and patience throughout the writing of this thesis. I am also grateful to my family, friends, and those closest to me for their constant support, positivity, and encouragement; this support was invaluable to me and essential for the completion of this thesis.

This work was also supported by the Ministry of Education, Youth and Sports of the Czech Republic through the e-INFRA CZ (ID:90254).

Abstract

Image classification is one of the core tasks in computer vision and is commonly addressed using Convolutional Neural Networks (CNNs), which are designed for grid-like data. Graph Neural Networks (GNNs), designed for graph data, offer an alternative approach by representing images as graphs and can be considered as a generalization of CNNs. The goal of this thesis is to review GNNs with a focus on their use in image classification, to select suitable datasets and architectures, and to compare CNNs and GNNs on this task. Images are converted into superpixel graphs using the SLIC algorithm and the Region Adjacency Graph (RAG) method, and the selected models include GCN and GraphSAGE for the GNNs and two custom CNN architectures. The experiments address four aspects: accuracy, trade-off between accuracy and computational cost, the effect of the number of superpixel segments, and per-class classification. The results show that CNNs achieve higher accuracy overall, while GNNs perform comparably only on simpler images but offer advantages in certain efficiency-related cases. The number of superpixel segments used in graph construction was shown to influence their performance. Overall, the thesis provides a theoretical background and an empirical comparison of CNNs and GNNs on image data across several evaluation criteria.

Keywords

artificial neural networks, deep learning, convolutional neural networks, graph neural networks, superpixels, image classification, Graph Convolutional Network, GraphSAGE

Contents

Introduction	6
1 Artificial Neural Networks	8
1.1 Convolutional Neural Networks	9
1.1.1 Architecture of CNNs.....	9
1.1.2 Depthwise separable convolution	11
2 Graph Neural Networks.....	12
2.1 Representation of images as graphs.....	12
2.1.1 Superpixel-graphs	12
2.2 Message passing framework	13
2.3 Architecture of GNNs for graph classification tasks.....	14
3 Application of GNN on Image Classification.....	17
3.1 Goals and Research Questions	17
3.2 Methodology	18
3.3 Datasets.....	19
3.4 Models.....	20
3.4.1 CNN model for CIFAR-10 dataset	22
3.4.2 CNN model for RESISC45 dataset.....	23
3.4.3 GCN model	25
3.4.4 GraphSAGE model.....	26
3.5 Implementation Details	27
3.6 Experiments.....	29
3.6.1 Comparison of CNNs and GNNs in accuracy	30
3.6.2 Comparison of CNNs and GNNs trade-off between accuracy and computational cost	30
3.6.3 Comparison of the different number of segments in the image-to-graph conversion	31
3.6.4 Analysis of per-class classification.....	32
4 Results and Discussion.....	33
4.1 Comparison of CNNs and GNNs in accuracy	33
4.2 Comparison of CNNs and GNNs trade-off between accuracy and computational cost	33
4.3 Comparison of the different number of segments in the image-to-graph conversion	34
4.4 Analysis of per-class classification.....	37

4.5 Discussion	41
Conclusions	44
List of references	46
Appendices	I
Appendix A: Electronic material	I

Introduction

Artificial neural networks (ANNs) have become a central area of machine learning, attracting attention as they can learn complex patterns in a wide range of tasks. They have many real-life applications in fields including healthcare, engineering, finance, insurance, transportation or media. Although the current popularity of ANNs is mostly driven by the transformer architecture, the earlier major breakthrough can be highly contributed to Convolutional Neural Networks (CNNs), which have been influential in the field of computer vision, specifically used for tasks such as image classification, image segmentation and object detection.

CNNs are tailored for grid-like data, such as images, and consistently achieve strong accuracies often even matching the human-level performance. These networks get their name from the convolution operation, where a filter slides over a small region of the image to pick up local patterns. CNNs have become the standard deep learning architecture to use for image data.

More recently, models that process non-Euclidian data have been gaining attention under the term geometric deep learning. Graph neural networks (GNNs), which belong to this area, are designed to process graphs – sets of nodes and edges. Graphs provide a convenient representation of complex real-life structures, such as social networks or chemical molecules but can also be derived from data that are not depicted as graphs by default. For images, one can think of each pixel as a node with connections to its neighboring pixels. There are also other methods of creation of graphs from images, such as extracting superpixels, that group similar regions of pixels, and connecting them.

Because images can be represented as graphs, a direct comparison between GNNs and CNNs on an image classification task becomes possible and is further motivated by the theoretical link between the two networks. Principles such as local connectivity and weight sharing used in CNNs also apply for GNNs and can be seen through the message-passing framework that most GNNs use, in which each node updates its embedding by aggregating information from its neighbors. This operation mirrors how CNNs combine features from local image regions, only the neighbors in graphs are represented by nodes and not pixels.

The goals of this thesis are to conduct a review of GNNs with a focus on their application in image classification, to select suitable datasets and architectures and compare CNNs and GNNs on an image classification task. Specifically, it asks: (i) how GNNs compare to CNNs in accuracy; (ii) how the two networks differ in trade-off between accuracy and computational cost; (iii) how the number of superpixel segments used to build graphs from images affects GNN metrics; and (iv) whether either network performs better for particular classes within the selected datasets.

The methodology includes a review of previous research on CNNs and GNNs for image classification, which was used to select suitable architectures and datasets. Next, it involves the preparation of the image datasets and their conversion to graphs, the implementation

of CNN and GNN architectures, the evaluation of the results, and lastly, the comparison of those results.

The comparison is conducted on the CIFAR-10 dataset as a standard benchmark and RESISC45 as a remote-sensing dataset with more complex scenes. The experiments employ the Graph Convolutional Network (GCN) and GraphSAGE as the GNN architectures. Images are converted to superpixel graphs using the SLIC algorithm, with edges formed by a Region Adjacency Graph (RAG). On the CNN side, two baseline architectures are used: a simple CNN with standard convolutions and a model with depthwise separable convolutions to reduce the parameter count for RESISC45 dataset.

The results of the experiments show that CNNs achieve higher accuracy on image classification tasks, while GNNs perform competitively on simpler images and offer advantages in certain efficiency-related aspects. The number of superpixel segments used in graph construction was shown to influence GNN performance.

The main contribution of this thesis lies in providing a comparison of CNNs and GNNs on image data that considers not only accuracy, but also computational cost, the influence of image-to-graph conversion parameters, and class-level analysis. Together with the theoretical background, which frames GNNs as a generalization of CNNs, this thesis aims to offer a comprehensive view of the topic.

Declaration on the Use of AI Tools: Generative AI tools were used to assist with writing code for visualizations and some of the modeling and evaluation functions and to stylistically refine some parts of the written text. All conceptual decisions, methodology, design of experiments, analyses, and interpretations were carried out independently by the author, and all AI outputs were reviewed, and adjusted where necessary.

1 Artificial Neural Networks

The name artificial neural networks (ANNs) comes from their inspiration by the biological brain, specifically the neurons (Goodfellow et al., 2016). ANNs imitate the neuron in its way of receiving, processing and sending information. The biological neuron receives signals by its dendrites, processes them in the cell body and pass some of the information to other neurons using its axons (Buduma et al., 2022). In an ANN, each artificial neuron receives input values, multiplies them by weights, sums them and passes the results through an activation function to produce an output (Buduma et al., 2022). Similarly to biological neurons where the strength of the connections between them changes according to how frequently they are used, the strength of connections between neurons in an ANN also changes during training with it adjusting its weights using backpropagation (Patterson & Gibson, 2017). Even though the largest modern ANNs do not approach the number of connections between neurons in the human brain (Patterson & Gibson, 2017), when used for tasks such as computer vision, ANNs outperform other kinds of machine learning algorithms and sometimes even match the human-level accuracies (Buduma et al., 2022).

Even though the underlying technology of ANNs dates back to the 1940s, the recent rising in popularity can be attributed to the increasing amount of training data and advancing hardware and software (Goodfellow et al., 2016). ANNs currently have many real-life applications and are popular in fields including healthcare, engineering, finance, insurance, transportation or media (Abiodun et al., 2018). Different types of networks are used based on their specific application. Core feed-forward networks, like the multi-layer perceptron, are used for fixed-size vector data, typically for the task of classification and regression. For sequential data, such as text, recurrent networks are used and for data that can be represented in a grid, like images, convolutional neural networks (CNNs) are utilized (Goodfellow et al., 2016). In particular for computer vision tasks, CNNs are among the most successful deep learning architectures (Bronstein et al., 2017) and serve as the primary network of choice for these tasks. It is, however, important to note that the introduction of the transformer architecture in 2017 (Vaswani et al., 2017) has not only made it the standard architecture for natural language processing tasks, but also led to its adaptation for computer vision tasks. Specifically for image classification tasks, the use of the transformer architecture has demonstrated comparable or better results in comparison to CNNs (Dosovitskiy et al., 2021).

Standard ANNs are designed for Euclidian data: fixed-size vectors, ordered sequences, and regularly shaped grids. In chapter 2 this fact will be revisited when discussing the graph neural networks (GNNs) which extend the deep neural models to non-Euclidian domains (Bronstein et al., 2017).

1.1 Convolutional Neural Networks

Convolutional neural networks (Lecun et al., 1998) were one of the biggest drivers that made the field of deep learning gain its popularity, with them highly outperforming other models in image-classification contests after the year 2011 (Aggarwal, 2018). They represent a specialized neural network for processing data with grid structure that have strong local spatial dependencies. Even though CNNs can be used for text-related tasks, as text can be thought of as a 1D grid, they are primarily used for image data, a 2D grid, and the related tasks such as image classification, image segmentation and object detection (Aggarwal, 2018).

The network was inspired by experiments on a cat's visual cortex which showed that mammals use different layers of cells in the visual cortex to construct parts of images at different levels of abstraction. This principle is used in CNNs as the hierarchical feature extraction, where in the early layers of the model simple shapes (such as the presence of edges at particular locations) are encoded and more complex shapes (motifs produced by the specific arrangements of edges, combination of those motifs etc.) are encoded in the later layers (Aggarwal, 2018; LeCun et al., 2015).

1.1.1 Architecture of CNNs

The name of the Convolutional neural networks stems from the mathematical operation called convolution that CNNs utilize in at least one layer (Goodfellow et al., 2016). However, most CNNs consist of multiple convolutional layers (Aggarwal, 2018). In a typical CNN each convolution is followed by a non-linear activation function and a pooling function is utilized to further modify the output and serves as a downsampling method (Goodfellow et al., 2016).

Convolutional layer

In the convolutional layer, the convolution operation is performed by placing multiple kernels (filters), usually of size 3x3 or 5x5, on each possible position of the input image and calculating a dot product between the kernel parameters and the values of the image pixels in their overlapping region.

Figure 1.1 Illustration of the convolution operation (source: Zhao et al., 2024)

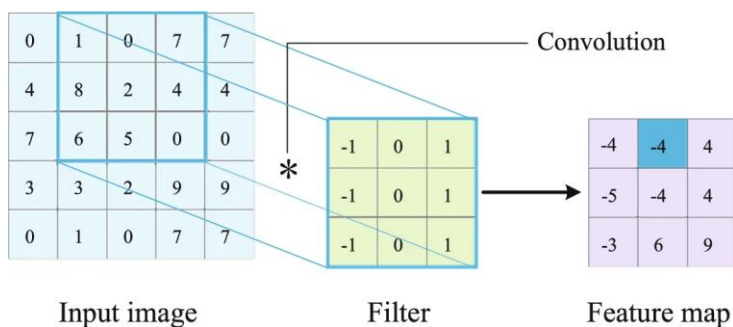


Figure 1.1 depicts the convolution operation in one position of the input image. The parameters of the kernels are learned during training and the output of this operation is a set of feature maps where each one indicates the presence of a specific spatial pattern (Aggarwal, 2018; Goodfellow et al., 2016).

Two important concepts that are used in the convolutional layer are local connectivity and weight sharing. Local connectivity means that during convolution, the kernel does not look at the whole image, but only at a local patch of the previous layer. Weight sharing implies that for each position in the image, the kernel uses the same set of parameters. That means that the same pattern is detected in each patch of the image (LeCun et al., 2015).

Activation functions

An activation function changes the output of a neuron given its input by introducing nonlinearity (Goodfellow et al., 2016). This enables neural networks to learn increasingly complex patterns especially as the number of layers grows (Aggarwal, 2018). Common activation functions include sigmoid, tanh and ReLU. Due to the simplicity (it outputs the input if it is positive, and zero otherwise) and fast performance in comparison to other activation functions, ReLU has been widely used in various neural networks models. ReLU also helps with solving the vanishing gradient problem, that the other mentioned activation functions struggle with. However, ReLU can suffer from the dying neuron problem but variants of ReLU such as leaky ReLU or parametric ReLU have been introduced to help mitigating it (Dubey et al., 2022).

Pooling layer

The process of pooling is responsible for the reduction of resolution of the image as it replaces a certain region of the image with a summary statistic. The most popular approaches include the use of max-pooling, which returns the maximum value in a square neighbor region of each feature map, or average pooling that returns the average instead of the maximum (Goodfellow et al., 2016). To reduce the spatial dimensions of the feature maps, pooling layers with 2x2 kernels and a stride of 2 are commonly used (Aggarwal, 2018).

The pooling operation can reduce memory needed to store parameters and contributes to translation invariance, allowing the network to focus on the presence of features rather than their exact position (Goodfellow et al., 2016).

Batch normalization and dropout

The method of batch normalization helps speeding up training by stabilizing the distributions of layer inputs through the normalization of their mean and variance (Santurkar et al., 2018). It also acts as a regularization technique, which helps with mitigating overfitting (Aggarwal, 2018). Another commonly used regularization method is dropout, which works by randomly deactivating (dropping) a subset of neurons during training, preventing the network from relying too heavily on specific activations. Both methods help the network to generalize better on unseen data (Patterson & Gibson, 2017).

Linear layer

The final linear layer combines the features extracted by the previous convolutional and pooling layers and maps them to the output space. For the task of classification, the linear layer produces the class scores (logits), which can then be passed through an activation function such as softmax, converting the scores to probabilities. Because the linear layers are fully connected, they account for most of the network's parameters. An approach that helps reduce the number of parameters is to apply average pooling before the linear layer which produces a single value (the spatial average) per feature map. The number of features fed into the final linear layer are therefore equal to the number of kernels (Aggarwal, 2018).

1.1.2 Depthwise separable convolution

Unlike the standard convolution, where each kernel is applied across all channels at the same time, depthwise separable convolution is a method that divides the convolution into two parts: depthwise convolution and a 1×1 pointwise convolution. Depthwise convolution applies a unique kernel for each input channel (for example to the three color channels (RGB) in an original input colorful image) and the 1×1 pointwise convolution then combines the outputs into one feature map. This approach significantly decreases the number of parameters and therefore reduces the computational cost and overall model size. Depthwise separable convolution gained popularity by its usage in architectures such as MobileNet and Xception (Zaidi et al., 2021).

2 Graph Neural Networks

As stated before, ANNs are designed for Euclidian data, but many real-life structures are more complex and are better represented using non-Euclidean data, such as graphs. Graphs, a set of objects (nodes) and connections between them (edges), are well suited for depicting relationships among its elements (Zhou et al., 2020) and in the domain of deep learning, they are often used to represent social networks, molecules, structures of the brain or gene expressions (Bronstein et al., 2017). The generalization of deep neural networks to non-Euclidean domains is referred to as geometric deep learning (Bronstein et al., 2017). Within this field, Graph Neural Networks (GNNs) are architectures specifically designed to operate on graph-structured data (Zhou et al., 2020).

There are three main learning tasks that the graph structure allows for: node-level, edge-level and graph-level. Node level tasks consists of node regression, node clustering or node classification which can be used, for example, to predict user attributes in social networks. Edge-level tasks focus on connections between the nodes and involve task such as edge classification and link prediction, like predicting the existence of a chemical bond between two atoms. The last type of common tasks are graph-level tasks, which include classification or regression of the whole graph. A common graph-level task is molecular property prediction (Tanis et al., 2024; Zhou et al., 2020).

In the application of GNNs to structures such as molecules or social networks, the graph structure is explicitly given. However, GNNs can be used on data for which the graph first has to be created, such as images (Zhou et al., 2020), therefore opening up the possibility of using GNNs for tasks including image classification formulated as a graph classification task.

2.1 Representation of images as graphs

Even though images are a typical example of grid-like data, when each pixel is treated as a node and connected to its eight neighboring nodes, images can be viewed as graphs with a regular grid-like structure. Features such as the color of the original pixel can be assigned to the nodes of the graph (Nikolentzos et al., 2021).

2.1.1 Superpixel-graphs

In many cases, representing each pixel as a graph node is unnecessary. Instead, we can group perceptually similar pixels into smaller regions, called superpixels, and thereby reduce the graph size and computational complexity (Nazir et al., 2021). Each superpixel then represents the node of the graph and can also be assigned multiple features.

Superpixel segmentation

There are several methods that can be used to convert images into superpixels, with SLIC being among the most popular within GNN literature (Nazir et al., 2021). SLIC, an abbreviation for Simple Linear Iterative Clustering, adapts k-means clustering algorithm which groups pixels in a 5D $labxy$ space of color and position. Three dimensions (lab) represent color in the CIELAB color space (l stands for lightness, a for green-red opponents and b for blue-yellow opponents) and the two other dimensions (xy) represent the position of the pixels. The SLIC algorithm therefore creates superpixels based on color similarity and spatial proximity. The benefits of this algorithm are that it produces compact and nearly uniform superpixels as well as its speed and storage efficiency (Achanta et al., 2012)

After the superpixels are created, they can have features assigned to them. This can be done by gathering all the pixels that belong to each superpixel and computing a desired summary statistic, such as the average color of that region. The average location of the superpixel can also be assigned by computing the centroid of the superpixel.

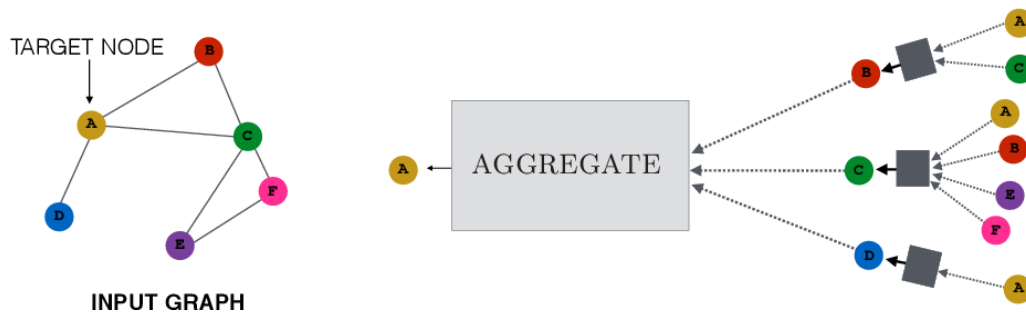
Edge creation

As with superpixel segmentation, there are several methods to create the graph edges, with the most common being the usage of a Region adjacency graph (RAG) or a K-Nearest Neighbors Graph (KNNG). In KNNG the edges are created by choosing k nearest neighbors of each node and connecting them. RAGs are created by simply connecting all directly adjacent superpixels therefore being less computationally complex than KNNGs (Nazir et al., 2021). The final graph can then be stored, for example, as an adjacency matrix.

2.2 Message passing framework

The message passing framework is a core principle used among the most popular GNNs. The main idea behind message passing is that at each iteration (layer), the node embeddings are updated based on the aggregated information (messages) from their neighboring nodes (Hamilton, 2020).

Figure 2.1 *Illustration of the message passing framework (source: Hamilton, 2020)*



Note: The illustration depicts a message-passing model with two layers

The message passing process can be expressed with two functions: AGGREGATE, which collects information from a node's neighbors, and UPDATE, which combines that information with the node's current embedding to generate its new embedding. Figure 2.1 illustrates the message passing process in a 2-layer model. In the first layer, for each node (e.g. A), the AGGREGATE function aggregates embeddings from its neighbors (B, C, D) to form a message. The UPDATE function, then combines this message with the node's previous embedding to produce a new embedding. In Figure 2.1, solid arrows denote the UPDATE step. In the second layer of the model, the same sequence of the two functions is applied again, but this time each neighbor's embedding already contains information from its own neighbors (e.g., B already incorporates signals from A and C). The new embedding produced by the UPDATE function therefore captures information from the 2-hop neighborhood (Hamilton, 2020). After k layers, the node embeddings would capture information about nodes k -steps away (Sanchez-Lengeling et al., 2021). The node embeddings produced by the message passing process generally capture information about the structure (e.g. how many connections a node has) and features of the node's neighborhood (e.g. similarity or contrast between a node's features and its neighbors) (Hamilton, 2020).

For a node-level task, these node representation are used for the final predictions but for a graph-level task, specifically graph classification, embedding of the whole graph is needed. This is obtained by using a readout function that aggregates the node representations. This readout function can be represented by functions such as summation or graph-level pooling functions (Xu et al., 2019).

The way that the node embedding capture the local aggregated features is analogous to how the convolution operation in CNNs aggregates features from local image regions. In GNNs this aggregation just happens over local graph neighborhoods (with the neighboring elements being nodes) rather than the spatial grids (with the neighboring elements being pixels) as seen in Figure 1.1 (Hamilton, 2020). Intuitively we can see, that the concept of local connectivity, that was introduced for CNNs, is also used in GNNs. Later, when the graph filtering layer will be discussed in chapter 2.3, it will be seen that the concept weight sharing that CNNs use is also satisfied in GNNs, as the same filter weights are reused across different nodes (Sanchez-Lengeling et al., 2021). It can be said that GNNs generalize convolution used in CNNs by preserving the principles of convolution. Unlike in an image, where the number of neighboring elements of a pixel is fixed, in graphs, the number of neighboring nodes can vary (Sanchez-Lengeling et al., 2021).

2.3 Architecture of GNNs for graph classification tasks

The general architecture of GNNs that are used for graph classification consists of the graph filtering layer, the activation layer (the use of the activation function) and the graph pooling layer. The graph filtering layers and the activation layers are generally stacked and used for generating node embeddings. The graph pooling layer follows them and is used for obtaining the embedding of the whole graph (Ma & Tang, 2021).

Graph filtering layer

The graph filtering operation is a process that takes the node features and the graph structure (e.g. the adjacency matrix) as input and produces updated node features. The operator that performs this process is referred to as the graph filter. The graph filters can be divided into spectral-based and spatial-based although some of the spectral-based filters can be viewed as spatial-based (Ma & Tang, 2021).

Spectral-based filters

Spectral-based approaches are based on graph signal processing and spectral graph theory and take advantage of the Graph Fourier Transform to express node features in the spectral (frequency) domain. In this view, node features that are similar in the adjacent nodes correspond to low frequency (smooth) graph signal, while sharp changes in the features correspond to high frequency. The spectral graph filter filters certain frequencies in the node features, so that some frequencies are amplified and some diminished, and just like in CNNs, the filter is learned (Ma & Tang, 2021).

An example of a network that uses the spectral-based filter is the Graph Convolutional Network (GCN) (Kipf & Welling, 2017). The filter in GCN is a simplified version of the spectral filter used in ChebNet, where ChebNet applies convolution through a localized polynomial filter that calculates the updated node representations from the node's k-hop neighborhood. The filter in GCN restricts this polynomial to $k=1$ and therefore only works with the 1-hop neighborhood (the directly adjacent nodes) (Ma & Tang, 2021). GCN is often referred to as a generalization of CNNs for graph data (Rodrigues & Carbonera, 2024, Xu et al., 2023). As indicated, the filtering in GCN can also be described with the spatial graph structure, where each layer updates a node by aggregating the representations of its neighbors while including the node itself in the aggregation. The GCN filter can therefore be viewed as a spatial-based filter (Ma & Tang, 2021).

Spatial-based filters

Spatial-based filters directly leverage the graph structure (the connections between nodes) and can be naturally expressed with the message-passing framework consisting of the aggregating and updating function.

A popular network that utilizes the spatial-based filter is GraphSAGE (Hamilton et al., 2017) which creates the node embeddings by randomly sampling and aggregating features from the node's neighborhood, hence the name "SAGE" that stands for Sample and Aggregate (Zhou et al., 2020). After aggregation, the neighborhood summary is combined with the node's own features to produce an updated node embedding. The sampling process can help with the issue of the growing number of neighboring nodes with each additional layer of the network, which is especially useful for processing large graphs (Hamilton et al., 2017, Ma & Tang, 2021). For the aggregating function the mean aggregator, LSTM aggregator or pooling aggregator can be utilized. The mean aggregator computes the element-wise mean of the neighboring node's features and outputs a single feature vector. The mean aggregator is similar to the filter in GCN, the difference is that the node's own features are concatenated with the average of the neighboring nodes whereas in the GCN filter, the computed average

includes the node itself. The LSTM aggregator handles the sampled neighboring nodes as a sequence with random ordering and use the LSTM architecture to process it. The pooling aggregator uses the max pooling operation to aggregate the features. For all of the mentioned aggregator options the aggregation happens only for the directly adjacent nodes (1-hop neighborhood) and the same aggregator is applied to every node (Ma & Tang, 2021).

The architecture of GNNs is also often categorized based on the used propagation module i.e. the module used to propagate information between nodes. Both of the mentioned architectures fall into the category of GNNs that use the convolutional operator, which is the most used operator among GNNs. The convolution operation is then implemented using the specified type of graph filter (spectral or spatial) (Zhou et al., 2020). These types of neural networks generalize the convolution operation known from CNNs to graph data by taking the weighted aggregation of a node's neighborhood information to generate the updated node representation (Wu et al., 2019). How much each neighbor contributes to the updated embedding is set by a weighting scheme defined by the given architecture. GCN, for example, weights neighbor contributions inversely proportional to node degrees, so nodes with many neighbors (high degree) receive smaller weight. GraphSAGE with the mean aggregator, on the other hand, treats all neighbors equally as no normalization is applied (Zhou et al., 2020).

Pooling Layer

The pooling layer is responsible for obtaining a representation of the whole graph by summarizing the information from the graph nodes. The pooling operation takes the node representations as input and outputs a new single node that represents the entire graph. In CNNs the two most common pooling operations were max-pooling and average pooling and both of these can be adapted to GNNs. These layers in GNNs apply the max operation or average operation to each feature dimension of the graph. There are also slightly more advanced pooling layers, called hierarchical, that reduce the graph only partially. Stacking these layers produces the graph representation by gradually reducing the graph at each stage, (Ma & Tang, 2021).

3 Application of GNN on Image Classification

This chapter focuses on the practical part of this thesis. It sets the goals and research questions, describes the methodology and implementation details. It also specifies the datasets and models used along with the reasons for their use. Finally the specific experiments are described.

3.1 Goals and Research Questions

The goals of this thesis are to conduct a review of Graph Neural Networks (GNNs) with a focus on their application in image classification, to select suitable image classification datasets and neural network architectures, and to apply GNNs and CNNs to selected image classification datasets and compare the results. The first goal was already satisfied in the two previous chapters. To satisfy the other goals, several research questions were set:

- 1. How do GNNs compare in accuracy to CNNs when applied to an image classification task and how do the results differ for different types of datasets?**

CNNs are the standard method for image classification tasks, while GNNs are a more recent approach allowing the modelling relationships between regions of an image. The results will provide an insight into whether GNNs can compete in terms of accuracy with CNNs on this task and what difference does the type of the dataset make.

- 2. How do convolutional and graph-based architectures compare in terms of the trade-off between test accuracy and computational cost?**

This question addresses the fact that while CNNs usually perform strongly on image classification tasks, their computational demands can grow with larger and higher-resolution datasets. Comparing the trade-off between test accuracy and computational cost with GNNs can show whether graph-based models could provide a more efficient alternative without a significant loss in accuracy.

- 3. How does the number of segments in the image-to-graph conversion method affect the metrics of GNNs?**

The performance of GNNs can depend on the way the images are transformed into graphs. The analysis will answer how does the number of segments in the SLIC

algorithm chosen during the conversion of images to superpixels affect the metrics of GNNs.

4. Do convolutional architectures or graph architectures work better for certain classes of selected datasets?

Different model families may have strengths on specific image classes. The goal of this question is to investigate whether there are visible patterns as to what classes do convolutional or graphs architectures work best for.

3.2 Methodology

1. Neural network architectures selection

The selection of neural network architectures was based on a survey of previous research on CNN and GNN use for image classification or other graph classification tasks.

The Graph Convolutional Network (GCN) (Kipf & Welling, 2017) was included as a baseline GNN architecture, which has been widely used in studies that apply GNNs on image data (Rodrigues & Carbonera, 2024; Nazir et al., 2021). The second GNN architecture GraphSAGE (Hamilton et al., 2018) was selected as a variant of GNNs that uses the spatial filter also mentioned in reviews (Xu et al., 2023) and studies that apply GNNs on image data (Nikolentzos et al., 2021).

For comparison with GNNs, a simple baseline CNN architecture was chosen along with a CNN using depthwise separable convolution chosen to reduce the number of parameters and computational cost.

2. Dataset selection

The datasets were selected based on a survey of previous research on the topic of image classification with GNNs. The first selected dataset was CIFAR-10, which serves as a baseline for image classification in multiple studies (Nazir et al., 2021; Rodrigues & Carbonera, 2024; Goyal & Steiner, 2022) and the second one was RESISC45, a dataset consisting of remote sensing images. This dataset was chosen after a review of available TensorFlow datasets to include one with more complex scenes, higher resolution and multiple object within each image.

3. Data preparation and conversion of images to graphs

Both datasets were preprocessed and in the case of GNN experiments, the image datasets were additionally converted to graphs. This was done by dividing the image into smaller regions called superpixels, which represent a group of perceptually similar pixels (Yang et al., 2020). Each superpixel then holds some information about the specific area of the image. In this work, the SLIC method was used.

Each superpixel was also assigned a set of features. In this work the features average color, centroid position and standard deviation from average color were chosen as these were the most influential to the performance of a GNN model in the work by

Rodrigues and Carbonera (2024). The created superpixels represent the nodes of the final graphs.

Next a graph is built by building edges between the nodes. In this work the RAG method was used, which adds an edge between all directly adjacent superpixels (Nazir et al., 2021). This creates the final graph with nodes and edges and eight node features, which can now be used as input for a GNN.

4. Model design and training process

Multiple neural network architectures were implemented. Both CNN and GNN models were trained using a unified training loop. The training function implemented early stopping, which stops the training after validation accuracy does not improve for a certain number of epochs. The best-performing model state was saved along with training summary metrics such as best validation accuracy, number of parameters, total training time and number of epochs.

5. Evaluation

The test splits across all models and datasets were evaluated by forwarding the graphs through the same network again. Metrics such as accuracy and recall are computed as well as confusion matrices with additional analysis of misclassification. Training time and inference time was also recorded to compare computational costs. All the evaluation metrics are explained in more detail for each experiment in chapter 3.6.

6. Result comparison

Finally, the results of all experiments were compared across the selected models and datasets.

3.3 Datasets

Two datasets were used in this work, CIFAR-10 and RESISC45, as shown in Figure 3.1. They were intentionally selected as they represent different domains and differ in complexity which allows to test CNNs and GNNs on datasets across varying characteristics and levels of difficulty for the image classification task.

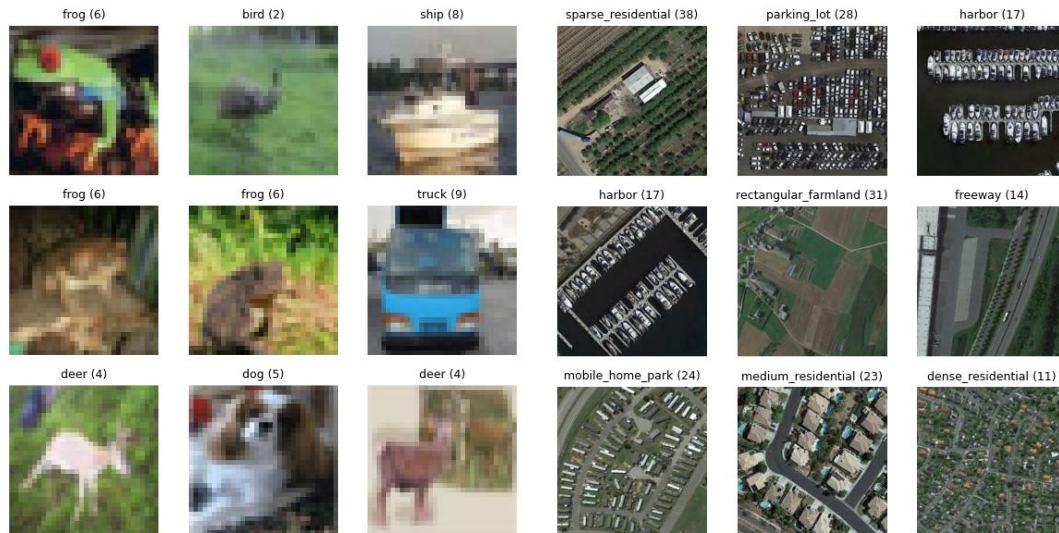
CIFAR-10¹ (Krizhevsky, 2009) represents a standard benchmark dataset for image classification. It was chosen for this work as it enables a comparison of performance between GNNs and CNNs on a dataset where CNNs are known to perform well. It was also used in multiple studies that used GNNs for an image classification task (Nazir et al., 2021; Rodrigues & Carbonera, 2024; Goyal & Steiner, 2022). CIFAR-10, named after the Canadian Institute for Advanced Research, which funded the project of creation of this dataset, is a subset of the 80 Million Tiny Images dataset and contains 60 000 color images in low

¹ <https://www.tensorflow.org/datasets/catalog/cifar10>

resolution (32x32 pixels) that are divided into 10 classes: airplane, automobile, bird, cat, deer, dog, frog, horse, ship and truck. The dataset is balanced, with 6000 images per each class. The objects on the images have relatively clear boundaries and usually do not interact with any other objects.

The RESISC45² (Cheng et al., 2017) dataset represents a more complex dataset. It is a publicly available remote sensing dataset created by Northwestern Polytechnical University and contains various scenes, including natural, rural and urban areas, capturing different spatial patterns. It contains 31 500 256x256 color images covering 45 classes with 700 images in each class. Since the dataset involves more complex scenes, often including multiple objects, it makes it possible to analyze their contextual relationship. This characteristic was one of the main reasons for selecting the RESISC45 dataset, as it gives GNNs the opportunity to capture the relationships between the objects. Another reason was the noticeably higher resolution of the images compared to CIFAR-10, which allows for a comparison of CNNs and GNNs on higher-quality images.

Figure 3.1 *Examples of the CIFAR-10 and RESISC45 datasets (source: Author)*



3.4 Models

For CNNs two separate architectures were developed each optimized for the individual dataset. For CIFAR-10, a simpler architecture consisting purely of convolutional layers was chosen, while for RESISC45 an architecture employing depth-wise separable convolution (which significantly reduced the number of parameters) was used.

Two GNN models were used for both datasets: Graph Convolutional Network (GCN) (Kipf & Welling, 2017) and GraphSAGE (Hamilton et al., 2018).

² <https://www.tensorflow.org/datasets/catalog/resisc45>

Both models fall under the category of models that use the convolutional operator (Zhou et al., 2020). The Graph Convolutional Network (GCN) was chosen as it can be understood as a generalization of CNN to graph data (Rodrigues & Carbonera, 2024) and therefore a great model for comparison with CNNs. The original model by Kipf & Welling was used for semi-supervised node classification task (Kipf & Welling, 2017), but by adding a pooling layer, it can be transformed for the task of graph classification. GraphSAGE can be seen as an extension of GCN and the SAGE in its name stands for Sample and Aggregate (Hamilton et al., 2017). Sample defines how neighboring nodes are chosen, and Aggregate how their embeddings are combined to update the node’s representation (Xu et al., 2023). The network was mainly designed for large graphs as the sampling helps with the problem of growing number of neighboring nodes for each additional layer (Zhou et al., 2020) but since the superpixel graphs in the used datasets are relatively small, the sampling step was not applied. The GraphSAGE network was chosen as it uses a purely spatial filter to serve as a comparison to GCN which is based on the spectral filter. Both architectures are not very deep and only consist of a few layers, which is standard for most state-of-the-art GNN models, as they usually have no more than 3-4 layers. This is because adding more layers leads to an over-smoothing problem where the nodes become indistinguishable (Zhou et al., 2023).

The decision to use separate CNN models for the datasets stems from the different nature of the image datasets themselves. They represent different domains and are also different in size. By converting the images into graphs, the input data was simplified to a similar level, so it becomes reasonable to use the same GNN architectures for both datasets.

Table 3.1 *Model training summary (source: Author)*

Dataset	Segments	Model	Trainable parameters	Best epoch	Val. accuracy	Avg. epoch time [s]	Training time [min]
CIFAR-10	—	Custom CNN	1 215 562	17	0.848	11.01	4.40
RESISC45	—	DSC	102 765	36	0.815	56.66	40.60
CIFAR-10	75	GCN	5 386	58	0.383	6.35	6.35
CIFAR-10	75	GraphSAGE	18 634	51	0.612	6.54	6.32
CIFAR-10	150	GCN	5 386	60	0.402	6.47	6.47
CIFAR-10	150	GraphSAGE	18 634	31	0.614	7.21	4.57
CIFAR-10	300	GCN	5 386	59	0.414	6.79	6.79
CIFAR-10	300	GraphSAGE	18 634	38	0.613	7.31	5.48
RESISC45	75	GCN	7 661	60	0.309	3.68	3.68
RESISC45	75	GraphSAGE	20 909	19	0.629	3.85	1.67
RESISC45	150	GCN	7 661	57	0.313	3.26	3.26
RESISC45	150	GraphSAGE	20 909	25	0.658	3.91	2.09
RESISC45	300	GCN	7 661	60	0.329	3.50	3.50
RESISC45	300	GraphSAGE	20 909	23	0.621	4.43	2.21

3.4.1 CNN model for CIFAR-10 dataset

The first CNN model is a rather simple CNN specifically designed for the CIFAR-10 dataset. It consists of six convolutional layers divided into three blocks. The number of filters is gradually increased to capture more complex information. The first block consists of three convolutional layers which first transform the input (3 channels) into 64 channels, then keep the same depth and then expand into 128 channels. Each convolutional layer is followed by batch normalization to stabilize the training and max pooling and dropout are applied after the third layer to reduce the size and prevent overfitting. The second block keeps the 128 channels and again applies batch normalization, max pooling and dropout. The third block then further increases the number of channels to 256 through two consecutive convolutional layers, each followed by batch normalization.

After the three convolutional blocks, the feature maps are adjusted into a fixed size by using the adaptive average pooling, in our case to size (1,1). The pooled output is flattened into a vector and passed through a fully connected linear layer and ReLU activation function. This is followed by a dropout. This vector is then passed to a fully connected linear layer for classification.

Code Listing 3.1 *Baseline CNN model for CIFAR-10 dataset (source: Author)*

```
class Custom_CNN(nn.Module):

    def __init__(self, num_classes, in_channels=3, pool_out=1, dropout=0.3):
        super().__init__()
        # Block 1
        self.conv1 = nn.Conv2d(in_channels, 64, kernel_size=3, padding=1)
        self.bn1 = nn.BatchNorm2d(64)

        self.conv2 = nn.Conv2d(64, 64, kernel_size=3, padding=1)
        self.bn2 = nn.BatchNorm2d(64)

        self.conv3 = nn.Conv2d(64, 128, kernel_size=3, padding=1)
        self.bn3 = nn.BatchNorm2d(128)
        self.pool1 = nn.MaxPool2d(2)
        self.drop1 = nn.Dropout(p=dropout)

        # Block 2
        self.conv4 = nn.Conv2d(128, 128, kernel_size=3, padding=1)
        self.bn4 = nn.BatchNorm2d(128)
        self.pool2 = nn.MaxPool2d(2)
        self.drop2 = nn.Dropout(p=dropout)

        # Block 3
        self.conv5 = nn.Conv2d(128, 256, kernel_size=3, padding=1)
        self.bn5 = nn.BatchNorm2d(256)

        self.conv6 = nn.Conv2d(256, 256, kernel_size=3, padding=1)
        self.bn6 = nn.BatchNorm2d(256)
```

```

        # Adaptive pooling to a fixed grid
        self.ap    = nn.AdaptiveAvgPool2d((pool_out, pool_out))
        self.flat  = nn.Flatten()
        self.fc1   = nn.Linear(256 * pool_out * pool_out, 256)
        self.drop3 = nn.Dropout(p=dropout)
        self.fc2   = nn.Linear(256, num_classes)

def forward(self, x):
    # Block 1
    x = F.relu(self.bn1(self.conv1(x)))
    x = F.relu(self.bn2(self.conv2(x)))
    x = F.relu(self.bn3(self.conv3(x)))
    x = self.pool1(x)
    x = self.drop1(x)

    # Block 2
    x = F.relu(self.bn4(self.conv4(x)))
    x = self.pool2(x)
    x = self.drop2(x)

    # Block 3
    x = F.relu(self.bn5(self.conv5(x)))
    x = F.relu(self.bn6(self.conv6(x)))

    # Head
    x = self.ap(x)
    x = self.flat(x)
    x = F.relu(self.fc1(x))
    x = self.drop3(x)
    logits = self.fc2(x)
    return logits

```

3.4.2 CNN model for RESISC45 dataset

For the RESISC45 dataset a model using the depthwise separable convolution was used. It starts with a standard convolution that uses stride=2 and therefore halves the size of the input image. Then four blocks of depthwise and pointwise convolution are applied. Each block consists of a depthwise 3x3 convolution (one filter per channel) and a pointwise 1x1 convolution. The filter sizes gradually expand. This design allows for a reduction of the number of parameters which is a great advantage for this higher resolution dataset. After each convolution batch normalization is applied to stabilize training and the ReLU activation function is used.

The head of the network applies global average pooling of size (1x1), uses dropout and a final fully connected linear layer to produce the classifications.

Code Listing 3.2 *Depthwise-separable CNN model for RESISC45 dataset (source: Author)*

```
class DSC(nn.Module):
    def __init__(self, num_classes, in_channels=3, dropout=0.3):
        super().__init__()

        # Stem (standard conv)
        self.conv1 = nn.Conv2d(in_channels, 32, kernel_size=3, stride=2,
                                padding=1)
        self.bn1 = nn.BatchNorm2d(32)

        # Block 1: Depthwise + Pointwise (stride=1)
        self.dw2 = nn.Conv2d(32, 32, kernel_size=3, stride=1, padding=1,
                                groups=32)
        self.bn2 = nn.BatchNorm2d(32)
        self.pw2 = nn.Conv2d(32, 64, kernel_size=1, bias=False)
        self.bn3 = nn.BatchNorm2d(64)

        # Block 2: Depthwise + Pointwise (stride=2)
        self.dw3 = nn.Conv2d(64, 64, kernel_size=3, stride=2, padding=1,
                                groups=64)
        self.bn4 = nn.BatchNorm2d(64)
        self.pw3 = nn.Conv2d(64, 128, kernel_size=1, bias=False)
        self.bn5 = nn.BatchNorm2d(128)

        # Block 3: Depthwise + Pointwise (stride=2)
        self.dw4 = nn.Conv2d(128, 128, kernel_size=3, stride=2, padding=1,
                                groups=128)
        self.bn6 = nn.BatchNorm2d(128)
        self.pw4 = nn.Conv2d(128, 192, kernel_size=1, bias=False)
        self.bn7 = nn.BatchNorm2d(192)

        # Block 4: Depthwise + Pointwise (stride=2)
        self.dw5 = nn.Conv2d(192, 192, kernel_size=3, stride=2, padding=1,
                                groups=192)
        self.bn8 = nn.BatchNorm2d(192)
        self.pw5 = nn.Conv2d(192, 256, kernel_size=1, bias=False)
        self.bn9 = nn.BatchNorm2d(256)

        # Head
        self.gap = nn.AdaptiveAvgPool2d(1)
        self.drop = nn.Dropout(p=dropout)
        self.fc = nn.Linear(256, num_classes)

    def forward(self, x):
        # Stem
        x = F.relu(self.bn1(self.conv1(x)))

        # Block 1
        x = F.relu(self.bn2(self.dw2(x)))
```

```

x = F.relu(self.bn3(self.pw2(x)))

# Block 2
x = F.relu(self.bn4(self.dw3(x)))
x = F.relu(self.bn5(self.pw3(x)))

# Block 3
x = F.relu(self.bn6(self.dw4(x)))
x = F.relu(self.bn7(self.pw4(x)))

# Block 4
x = F.relu(self.bn8(self.dw5(x)))
x = F.relu(self.bn9(self.pw5(x)))

# Head
x = self.gap(x).flatten(1)
x = self.drop(x)
logits = self.fc(x)
return logits

```

3.4.3 GCN model

The first GNN used was a simple network based on Graph Convolutional Network (GCN). It consists of two GCN layers, each transforming the input features into a 64-dimensional vector by incorporating information from the neighboring nodes. Each layer is followed by the ReLU activation function and dropout to prevent overfitting. After the two convolutional layers, global mean pooling is applied. This pooling obtains the graph-level embeddings by computing the average of node embeddings for each graph, which is needed for performing the graph classification task. The pooled vector with these embeddings is then passed to a fully connected linear layer for classification.

Code Listing 3.3 *GCN model (source: Author)*

```

class Simple_GCN(nn.Module):
def __init__(self, in_channels, hidden_channels=64, out_channels,
              dropout=0.3):
    super().__init__()
    self.conv1 = GCNConv(in_channels, hidden_channels)
    self.conv2 = GCNConv(hidden_channels, hidden_channels)
    self.lin = nn.Linear(hidden_channels, out_channels)
    self.dropout = dropout

def forward(self, x, edge_index, batch):
    # Block 1
    x = self.conv1(x, edge_index)
    x = F.relu(x)
    x = F.dropout(x, p=self.dropout, training=self.training)

    # Block 2

```

```

x = self.conv2(x, edge_index)
x = F.relu(x)
x = F.dropout(x, p=self.dropout, training=self.training)

# Aggregate to graph-level representation
x = global_mean_pool(x, batch)
return self.lin(x)

```

3.4.4 GraphSAGE model

The second GNN model utilizes the GraphSAGE framework. It consists of three convolutional layers, each using mean for the aggregation function. This way the nodes update their features by using the average of the feature vectors of their neighboring nodes. Batch normalization is applied after every convolutional layer to stabilize training, followed by a ReLU activation function. Dropout is also applied after the first and third layer to reduce overfitting. Same as in the GCN model, after all convolutional layers, global mean pooling is applied and followed by a final fully connected linear layer.

Code Listing 3.4 *GraphSAGE model (source: Author)*

```

class GraphSAGE(nn.Module):
    def __init__(self, in_channels, hidden_channels=64, out_channels,
                  dropout=0.3):
        super().__init__()
        self.conv1 = SAGEConv(in_channels, hidden_channels, aggr='mean')
        self.bn1 = BatchNorm(hidden_channels)

        self.conv2 = SAGEConv(hidden_channels, hidden_channels, aggr='mean')
        self.bn2 = BatchNorm(hidden_channels)

        self.conv3 = SAGEConv(hidden_channels, hidden_channels, aggr='mean')
        self.bn3 = BatchNorm(hidden_channels)

        self.lin = nn.Linear(hidden_channels, out_channels)
        self.dropout = dropout

    def forward(self, x, edge_index, batch):
        # Block 1
        x = self.conv1(x, edge_index)
        x = self.bn1(x)
        x = F.relu(x)
        x = F.dropout(x, p=self.dropout, training=self.training)

        # Block 2
        x = self.conv2(x, edge_index)
        x = self.bn2(x)
        x = F.relu(x)

```

```

# Block 3
x = self.conv3(x, edge_index)
x = self.bn3(x)
x = F.relu(x)
x = F.dropout(x, p=self.dropout, training=self.training)

# Aggregate to graph-level representation
x = global_mean_pool(x, batch)
return self.lin(x)

```

3.5 Implementation Details

The code was written in Python mainly using the *Pytorch* framework (Paszke et al., 2019), leveraging its *Torchvision*³ library for the loading of the datasets and *Pytorch Geometric* for modeling of the neural networks. The machine learning library *scikit-learn* was also used during the creation of superpixel-graphs. Due to the higher computational demand (especially of the conversion of images to graphs and the training process), GPU provided by the MetaCentrum infrastructure was used, made available through the e-INFRA project. The Python notebook with all the code along with all files that were saved throughout the conduction of experiments is included in Appendix A: Electronic material.

Loading datasets, preprocessing

The CIFAR-10 dataset was loaded directly from *Torchvision* using the `torchvision.datasets` module. As the RESISC45 dataset is not part of the built-in datasets in *Torch*, it was downloaded manually, in our case saved to a github repository, and then loaded using *torchvision*. During this process both the datasets were also preprocessed by using `transforms.ToTensor()` class which converts the images from shape (Height x Width x Channels) to (Channels x Height x Width). This was done to make the images compatible with *Pytorch*. This class also normalized the pixel values from range (0,255) to range (0,1). Both datasets were split into train, test and validation sets using the 80:10:10 ratio, loaded into *PyTorch Geometric* data loaders which split them into batches with 32 samples per batch and made sure that GPU would be used for training if available. Shuffling was also applied but only to the train sets and with a set seed that ensures reproducibility.

Superpixel-graphs creation

The conversion to superpixels was done using the SLIC method by implementing the `slic`⁴ algorithm from *scikit-image* library. This function takes 2 input parameters – the number of desired cluster centers (`n_segments`) and the compactness factor (`compactness`). The compactness parameter changes the tradeoff between color proximity and spatial

³ <https://docs.pytorch.org/vision/stable/index.html>

⁴ <https://scikit-image.org/docs/0.25.x/api/skimage.segmentation.html#skimage.segmentation.slic>

proximity. This parameter can be in the range from 1-40 and the larger it is, the more important spatial proximity is and the lower it is, the more important color proximity is (Achanta et al., 2012). In our case the compactness parameter was set to 10. For the `n_segments` parameter three values were used – 75, 150 and 300. The images in both datasets were therefore highly reduced as the resulting image sizes were around 75, 150 or 300 superpixels per image (depending on the settings of `n_segments`) which is a significant difference from the original sizes of 1024 pixels for the CIFAR-10 dataset ($32 \times 32 = 1024$) and 65 536 pixels in case of the RESISC45 dataset ($256 \times 256 = 65\,536$). It is important to note that the specific number of `n_segments` in the slic algorithm does not always yield the exact number of superpixels (Rodrigues & Carbonera, 2024).

After superpixels were created, each superpixel was assigned the features average color and standard deviation from average color (R,G,B) along with the centroid position.

The edges between the superpixels (which now represent the graph nodes) are built using the `rag_mean_color` function from the *scikit-image* library which connects the adjacent superpixels. It also computes the edge weight using the difference between the mean colours of the superpixels, but the edge features are not further used in this work, as we only work with node features.

It is also important to note that any images with less than 2 superpixels were skipped as it would be impossible to create the RAG. This has happened for two images from the train set and one image in the validation set of RESISC45 in the case of segmentation to 75 segments. With 150 and 300 segments the slic method was able to find more superpixels, so it was possible to create a graph.

The final graphs were then saved and loaded into data loaders.

Modeling

Modeling in *Pytorch* is organized into two parts, the training functions and the model classes. Two training functions were implemented. One function runs a single epoch and computes the cross-entropy loss. If it is in training mode, it also performs backpropagation and updates the model parameters. The other function then handles the entire training process by calling the first function repeatedly and tracking the performance metrics and timing. It also stores best-performing model weights.

The models themselves were implemented as separate classes. Each model class has two methods: `__init__` which defines the specific layers and *forward* that specifies how the data pass through these layers.

For the CNN models, the parameter for the forward method is the image itself whereas with GNNs, the parameters are the feature matrix, edge index and the batch. The feature matrix⁵ has shape: `x=[num_nodes, num_node_features]`. The number of node features is eight as specified earlier (we have features: mean R, mean G, mean B, standard deviation of R, G

⁵ https://pytorch-geometric.readthedocs.io/en/latest/generated/torch_geometric.data.Data.html

and B, mean centroid x, mean centroid y) and the edge index⁵ has shape `edge_index=[2, num_edges]` and describes the graph connectivity. Graph connectivity is stored in COO (coordinate list) and contains two lists – first list stores the source nodes and the second one stores the destination nodes. It is an alternative form to the adjacency matrix that is more memory-efficient when dealing with sparse matrices.

3.6 Experiments

This chapter describes the design of the experiments that were conducted to answer the research questions. For all the experiments the CIFAR-10 and RESISC45 datasets were used. To ensure consistency, the same training and evaluation settings were applied across all experiments.

CNNs

- Two custom architectures were used, each tailored specifically to each dataset. A simple CNN with expanding number of filters for the CIFAR-10 dataset and a Depthwise-separable CNN for the RESISC45 dataset.
- The input for both CNN models were the original images.

GNNs

- Two architectures were used for both datasets with the same setting. One architecture was based on the Graph Convolutional Network (GCN) and the other one on the GraphSAGE framework.
- The input for the GNN models were the graphs created from the images using superpixel segmentation (with SLIC method) and edge creation (with RAG method).
- The node features were kept the same for all datasets and consisted of eight features in total: mean and standard deviation of RGB values and centroid coordinates.

Training setup for CNNs and GNNs

- Optimizer: Adam with learning rate 0.001 and weight decay=0.0005
- Loss function: cross-entropy
- Early stopping: monitored on validation accuracy, patience of 7 epochs
- Dropout: 0.3
- Number of epochs: 60
- Hyperparameter selection: selected using trial and error for each model separately

Evaluation metrics

- Different evaluation metrics were used for different experiments which are further stated in the subchapters for each experiment. In total, these metrics were used for evaluation of the experiments:
 - Training accuracy
 - Testing accuracy

- Inference time
- Training time
- AccuracyInference Efficiency, AccuracyTraining Efficiency (further described in chapter 3.6.2)
- Conversion time to superpixel-graphs
- Per-class recall
- Multi-class confusion matrix

3.6.1 Comparison of CNNs and GNNs in accuracy

The purpose of this experiment is to directly compare the performance of CNNs and GNNs on different datasets. The evaluation was done using test accuracy which describes the proportion of correct predictions over the total number of predictions.

$$Accuracy = \frac{Number\ of\ correct\ predictions}{Number\ of\ all\ predictions} \quad (3.1)$$

Accuracy was chosen as it is among the most common evaluation metrics to use for a classification task and since the classes in both datasets are balanced, metrics such as F1-score are not implemented.

It is expected that the CNN models will perform better in terms of accuracy as they are specialized for working with grid-like structure data, especially with images (Goodfellow et al., 2016).

3.6.2 Comparison of CNNs and GNNs trade-off between accuracy and computational cost

This experiment was conducted to assess the efficiency of CNNs in comparison to GNNs. The purpose is to examine whether GNNs can provide a better balance of accuracy and cost than CNNs. For evaluation, the cost was represented by the inference time, i.e. the total time it takes the model to classify the unseen test split, and by training time. The trade-off between accuracy and inference time was computed by using formula 3.2. The higher the value, the better the model is in balancing accuracy and inference time.

$$AccuracyInference\ Efficiency = \frac{Test\ accuracy}{Inference\ time} \quad (3.2)$$

To assess not only how efficiently the models predict, but also how efficiently they learn, the training time was considered. The ratio of test accuracy to total training time was therefore computed as shown in formula 3.3. Higher values denote models that achieve good accuracy faster.

$$AccuracyTraining\ Efficiency = \frac{Test\ accuracy}{Training\ time} \quad (3.3)$$

The expectations for this experiment are, that both the trade-off between accuracy and inference time and accuracy and training time will be better for the GNN models as the input for the GNNs is much smaller then the original image and therefore both the training and predictions can be done faster.

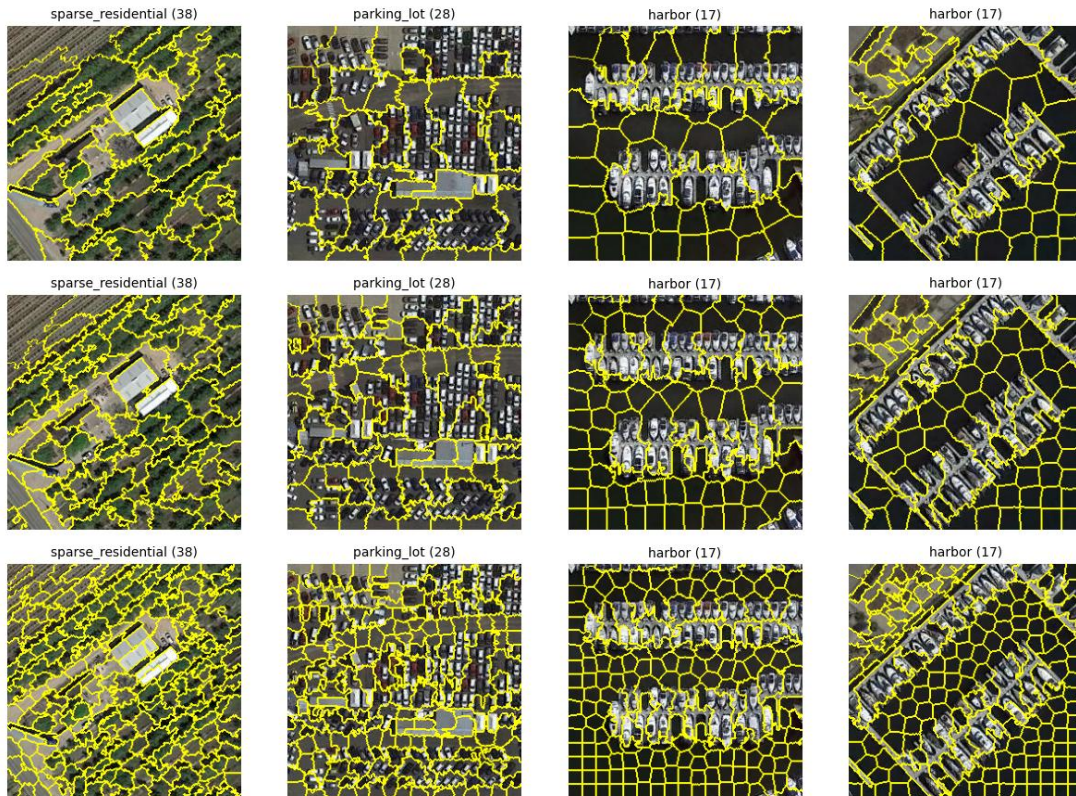
3.6.3 Comparison of the different number of segments in the image-to-graph conversion

This experiment examines the sensitivity of GNNs to the choice of the number of segment in the image-to-graph conversion. Three values have been tried as the number of parameters in the slic algorithm – 75, 150 and 300. Examples of the superpixels generated using different number of segments can be seen in Figure 3.2. The superpixels were connected using the RAG method into superpixel-graphs. All six graph datasets (three different setting for two datasets) were fed to the GNNs.

The experiment was evaluated using accuracy, training time, inference time and also the time it took to convert the image datasets to graphs.

The expected results are that the accuracy will get slightly better with the increased number of superpixels, as the graph then holds more information about the original image. The training time and inference time are expected to get slightly longer with the increasing number of superpixels as is the time it takes to convert the images to superpixel-graphs. This is because the larger the graph is, the longer it might take to create it and process it.

Figure 3.2 *Examples of superpixels of RESISC45 dataset generated using 75, 150 and 300 segments (source: Author)*



3.6.4 Analysis of per-class classification

The purpose of this experiment is to investigate whether selected CNN and GNN models perform better for specific classes of the CIFAR-10 and RESISC45 datasets. The motivation was to analyze class-level differences that are not visible in the overall accuracy. For this experiment, all the datasets and models were used.

For evaluation the per-class recall was computed along with the multi-class confusion matrix both accompanied with heatmap visualizations. In its standard form, recall is the proportion of correctly classified actual positives over all the actual positives. The recall computed per each class then denotes the class instances that were correctly classified out of all instances of that class. Formula 3.2 shows its computation for class i .

$$Recall_i = \frac{True\ Positive_i}{True\ Positives_i + False\ Negatives_i} \quad (3.4)$$

Multi-class confusion matrix values were also used to evaluate this experiment. This matrix has the true classes on the y-axis and the predicted classes on the x-axis. The diagonal then shows how many images were classified correctly while the off-diagonal values display how many images were misclassified into which class. For this experiment only the off-diagonal values were used, as there is focus on the misclassifications. The values were also normalized to get a better understanding of the overall patterns.

4 Results and Discussion

The following sections present the main findings of the conducted experiments. A discussion chapter follows in which the experiments are linked to the research questions and their results are interpreted.

4.1 Comparison of CNNs and GNNs in accuracy

This experiment compared the CNN and GNN models in accuracy to measure their ability to correctly classify the input data. The results of this experiment are shown in Table 4.1.

Table 4.1 *Model accuracy summary for best performing models (source: Author)*

Dataset	Segments	Model	Train Accuracy	Test Accuracy
CIFAR-10	—	Custom CNN	0.835	0.845
RESISC45	—	DSC	0.793	0.804
CIFAR-10	300	GraphSAGE	0.608	0.610
RESISC45	150	GraphSAGE	0.635	0.650

The CNN models achieved the highest overall accuracies on both datasets, outperforming the GNN models by 0.235 for CIFAR-10 and 0.154 for RESISC45. These results follow the expectations for this experiment and confirm that CNNs are very well suited for the task of image classification in terms of achieving great accuracy.

Among the GNN models, GraphSAGE achieved better results than the GCN model, outperforming it on both datasets. The performance of GraphSAGE on the RESISC45 dataset was better in comparison the the CIFAR-10 dataset.

Neither of the best performing models had substantial problems with overfitting as can be seen by looking at their train accuracy in comparison with the test accuracy.

4.2 Comparison of CNNs and GNNs trade-off between accuracy and computational cost

To compare CNNs and GNNs in terms of accuracy and computational cost, the accuracy per cost was measured, where cost was represented by inference time and training time. Two metrics, AccuracyInference and AccuracyTraining, were computed, as defined previously.

Table 4.2 *Trade-off between accuracy and computational cost for models with best accuracy (source: Author)*

Dataset	Segments	Model	Trainable parameters	Test Accuracy	Inference time [s]	Training time [min]	AccuracyInference Efficiency	AccuracyTraining Efficiency
CIFAR-10	—	Custom CNN	1 215 562	0.845	0.343	4.40	2.464	0.192
RESISC45	—	DSC	102 765	0.804	0.504	40.60	1.595	0.020
CIFAR-10	300	GraphSAGE	18 634	0.610	0.330	5.48	1.848	0.111
RESISC45	150	GraphSAGE	18 634	0.650	0.103	2.09	6.311	0.311

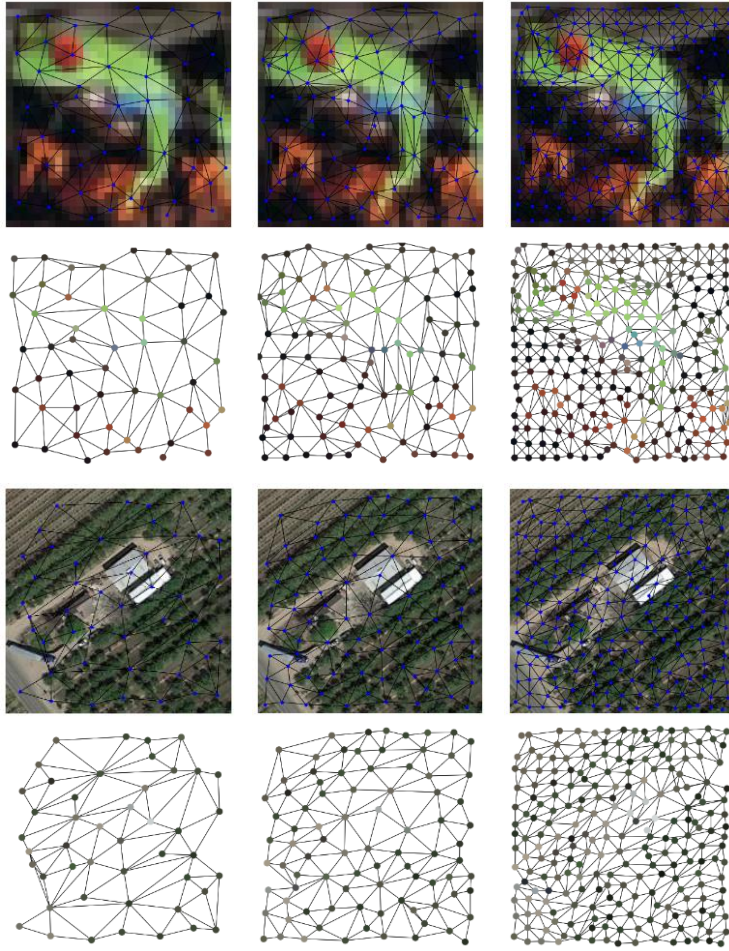
The results in Table 4.2 show that for the CIFAR-10 dataset, the CNN model performed better in terms of AccuracyInference Efficiency than the corresponding GNN model. Nevertheless it is important to state that the efficiency was only about 1.3 times higher. On the other hand the results for the RESISC45 dataset show a visible difference in the AccuracyInference Efficiency metric as the GNN model’s efficiency is almost 4 times higher than the CNN model’s. Hence even though the GNN model’s accuracy is lower, it shows notably better results in terms of the trade-off between accuracy and inference time.

The AccuracyTraining Efficiency of both CIFAR-10 and RESISC45 showed similar results. For CIFAR-10, the CNN model was about 1.7 times more efficient than the corresponding GNN model. For RESIC45 the GNN model also performed better in terms of the AccuracyTraining efficiency. The efficiency was even higher in this case, as the GNN model trained almost 20 times faster. Although its accuracy does not compare to the one of the CNN model, it was still more than 15 times as efficient.

4.3 Comparison of the different number of segments in the image-to-graph conversion

This experiment evaluated how the number of desired superpixels used in the SLIC algorithm affects both the computational cost and accuracy. Three segmentation values were tested (75, 150 and 300) for both datasets. Examples of the superpixel graphs can be seen in Figure 4.1.

Figure 4.1 Examples of superpixel-graphs of CIFAR-10 and RESISC45 dataset generated using 75, 150 and 300 segments (source: Author)



Note: Rows 2 and 4 display graphs with nodes colored by the average superpixel color

Table 4.3 demonstrates that the number of segments directly impacts the time it takes to convert the images to superpixel graphs as with the increasing number of segments the time also increases. For CIFAR-10 the conversion time increased approximately in proportion to the number of segments while the conversion time for RESISC45 was much more flatter. Across all three configurations, the RESISC45 conversion time was substantially higher, reaching about 3 hours and 18 seconds for the highest number of segments.

Table 4.3 Conversion time to superpixel graphs for different number of segments (source: Author)

Dataset	Segments	Average number of nodes	Conversion time to superpixel-graphs
CIFAR-10	75	53.32	12 min 34 s
CIFAR-10	150	108.69	20 min 2 s
CIFAR-10	300	239.77	41 min 5 s
RESISC45	75	57.39	2 h 39 min 9 s

RESISC45	150	111.30	2 h 49 min 42 s
RESISC45	300	234.41	3 h 17 min 58 s

Table 4.3 also shows that the desired number of segments parameter in the SLIC algorithm does not guarantee that exact segmentation. The average number of nodes that the algorithm produces is lower than the desired number in all cases. This is caused by the nature of the image itself. During the segmentation into superpixels for this experiment, there were three cases where the SLIC algorithm did not find any superpixels. This specifically occurred for two images in the train split and one image in the validation split of the RESISC45 dataset for 75 desired segments. The problematic images are shown on Figure 4.2. The texture of the images is so busy that the algorithm had problems with finding similar regions. For the division into 150 and 300 segments, the algorithm was able to find several superpixels and the graph could therefore be constructed.

Figure 4.2 Problematic images for SLIC segmentation from the RESISC45 dataset (source: Author)



Table 4.4 Model metrics for different number of segments (source: Author)

Dataset	Segments	Model	Test Accuracy	Inference time [s]	Training time [min]
CIFAR-10	—	Custom CNN	0.845	0.343	4.40
RESISC45	—	DSC	0.804	0.504	40.60
CIFAR-10	75	GCN	0.384	0.326	6.35
CIFAR-10	75	GraphSAGE	0.608	0.324	6.32
CIFAR-10	150	GCN	0.410	0.326	6.47
CIFAR-10	150	GraphSAGE	0.609	0.325	4.57
CIFAR-10	300	GCN	0.418	0.337	6.79
CIFAR-10	300	GraphSAGE	0.610	0.330	5.48
RESISC45	75	GCN	0.296	0.102	3.68
RESISC45	75	GraphSAGE	0.615	0.103	1.67
RESISC45	150	GCN	0.309	0.102	3.26
RESISC45	150	GraphSAGE	0.650	0.103	2.09
RESISC45	300	GCN	0.321	0.105	3.50
RESISC45	300	GraphSAGE	0.604	0.103	2.21

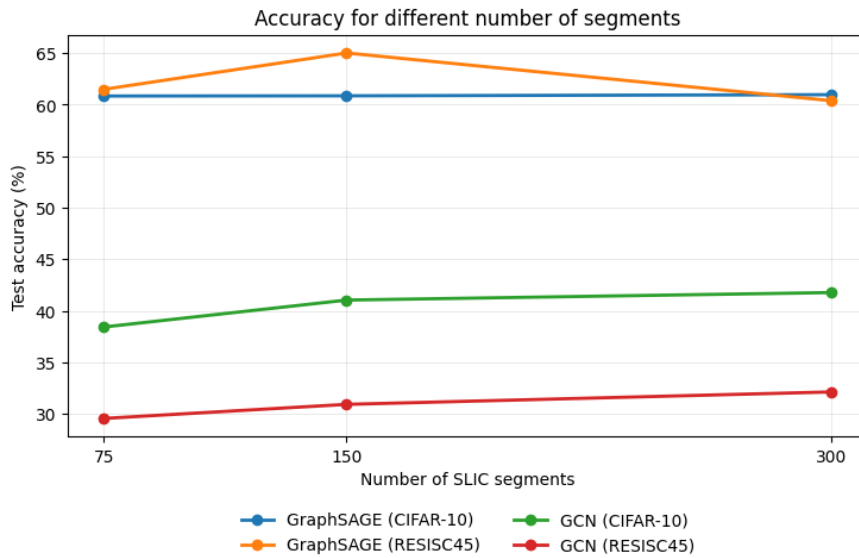
Note: CNN and GNN models with best test accuracies for each dataset are highlighted

In terms of accuracy there are different patterns for each dataset as can be seen in Table 4.4. and Figure 4.3. For CIFAR-10 the GraphSAGE model outperformed the GCN model on all segmentation levels but the accuracy improvements for increasing number of segments were only minimal. The GCN model achieved much worse results in comparison but the model got gradually better with the increasing number of segments. The best accuracy on CIFAR-10 was 0.610 and was achieved by GraphSAGE with the desired number of segments set to 300. For RESISC45 the GCN model was also outperformed by GraphSAGE and the accuracy improved when progressing from 75 segments to 150. However, the GraphSAGE model showed accuracy degradation after 150 segments, where the best accuracy of 0.650 was achieved.

In terms of inference time, the choice of number of segments did not make a significant change. The biggest difference in inference time across both models and all segments is 13 milliseconds for CIFAR-10 and only 3 milliseconds for RESISC45.

The effect of the number of desired segments on training time was also minor for both datasets. For CIFAR-10, the training time of GCN model remained relatively stable around 6 minutes, while GraphSAGE ranged between 4.57 min and 6.32 min, showing no consistent increase with the number of segments. For RESISC45 the training time was generally faster. The GCN model also had stable training times, specifically around 3 minutes and GraphSAGE also showed no consistent increase pattern.

Figure 4.3 Accuracy for different number of segments (source: Author)



4.4 Analysis of per-class classification

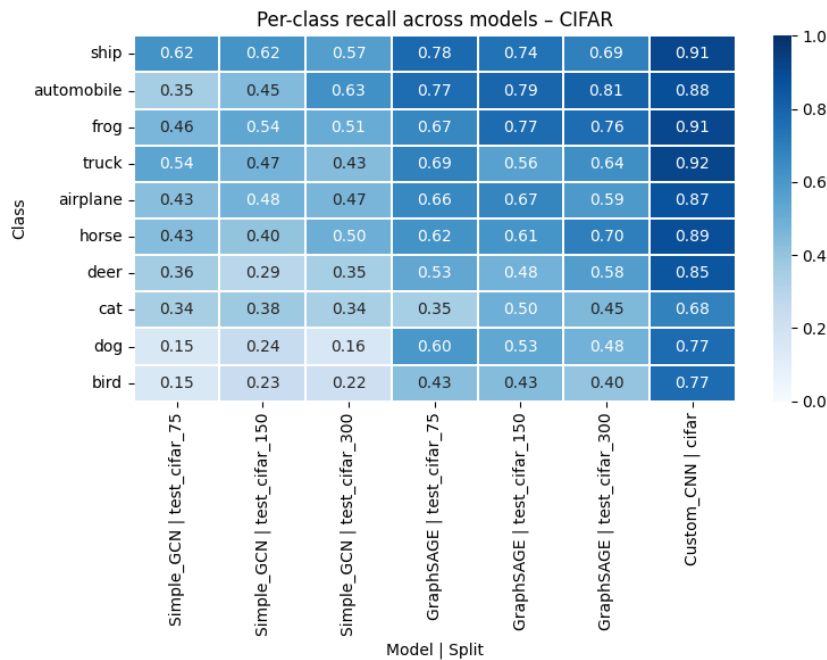
The previous experiments focused on overall metrics, while this section explores the class-level behavior, specifically the recall values and the most common misclassifications.

CIFAR-10

Figure 4.4 shows the recall for each class of the dataset across both models and all segmentation settings. As recall measures the number of instances correctly classified among all occurrences, the higher the per-class recall is, the better the model is at correctly classifying that class, while low recall means that the model often misclassifies that class. The CNN model showed by far the best recall result for all classes, with the lowest value being 0.68 for the category *cat*. The two GNN models both had worse results and showed a noticeable difference between each other, with GraphSAGE being better of the two across all classes. On average the class with the least misclassifications across all models, datasets and segments is *ship* and with the most misclassifications is *bird*. The increasing number of segments did not show any major pattern in the per-class recall values.

From the GNN models, GraphSAGE achieved comparable results with CNN only for the class *automobile*. Even though GraphSAGE outperformed GCN in all classes, classes such as *bird*, *dog*, *cat* and *deer* were the most challenging for both GNN models.

Figure 4.4 Per-class recall across models for CIFAR-10 (source: Author)



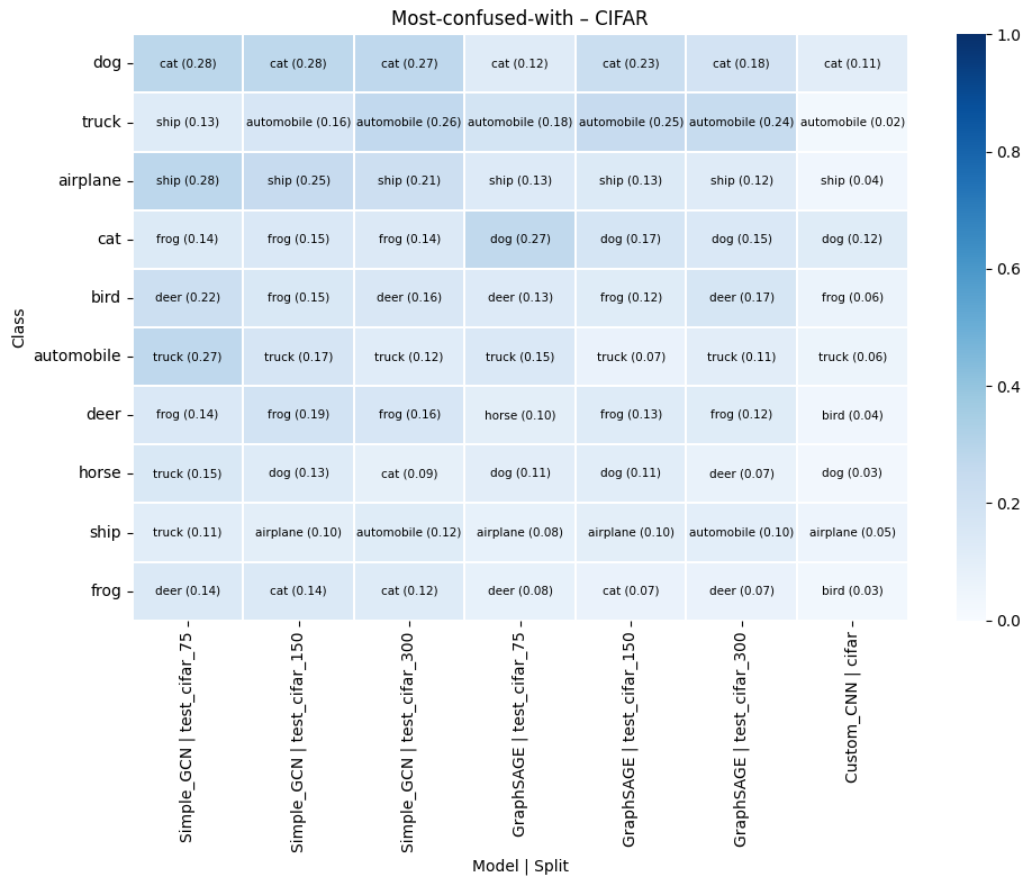
Note: Classes are sorted by their average recall across all models

Next, the classes with which the models most frequently confused the true labels were examined as seen in Figure 4.5. Both GNN and CNN models showed most confusion between visually similar or contextually related categories, such as *dog-cat*, *airplane-ship* and *truck-automobile*. The CNN model achieved clearly stronger class separation, with lower confusion values across all categories. The biggest confusion was for the already mentioned *dog-cat* pair, where 11 % of all images with dogs were incorrectly predicted as cats and 12 % of images with cats were predicted as dogs. All other biggest misclassifications of the CNN model fall under 10 %. For the GNN models, GraphSAGE either got similar results as the GCN and in some classes (e.g. *truck-automobile* or *dog-cat*) outperformed it.

For both GNN models, increasing the number of segments slightly altered the most confused-with classes but it did not show a clear pattern of significantly lowering the confusion values. In some cases (e.g. *truck-automobile* for the GCN model), confusion even rises with more segments.

Overall it can be said that the GNN models often mirror the most prominent misclassifications of the CNN model, with GraphSAGE generally performing better than GCN but still falling behind the CNN model.

Figure 4.5 *Most-confused-with classes with the normalized confusion matrix value for CIFAR-10 (source: Author)*



RESISC45

The per-class recall analysis for RESISC45 presented on Figure 4.6 indicates that both GNN and CNN models achieve the best performance on natural scenes such as *snowberg*, *desert*, *meadow*, *forest* and *sea ice* while struggling more with the urban scenes with specific buildings or roads. On average among the classes that were most misclassified across all models are *basketball court*, *church*, *stadium*, *palace* or *roundabout*. There is a visible difference between all models, with the CNN model having the highest recall across nearly all classes. Among the GNNs, GraphSAGE again outperformed GCN, this time almost in all classes. In other natural scenes such as *island*, *circular farmland*, *cloud*, *lake*, *golf course* and *mountain* as well as in the previously mentioned natural scenes where all models performed well, GraphSAGE even approached CNN-level recall.

Both GNNs, especially graphSAGE, performed close to CNN on larger, visually clearer natural landscapes. The CNN model however remained superior in classes that depend on finer local details (such as *golf course*, *tennis court* or *baseball diamond*).

As with CIFAR-10 the number of segments had little effect on the recall for each class, sometimes lowering it and sometimes increasing it.

Figure 4.6 Per-class recall across models for RESISC45 (source: Author)

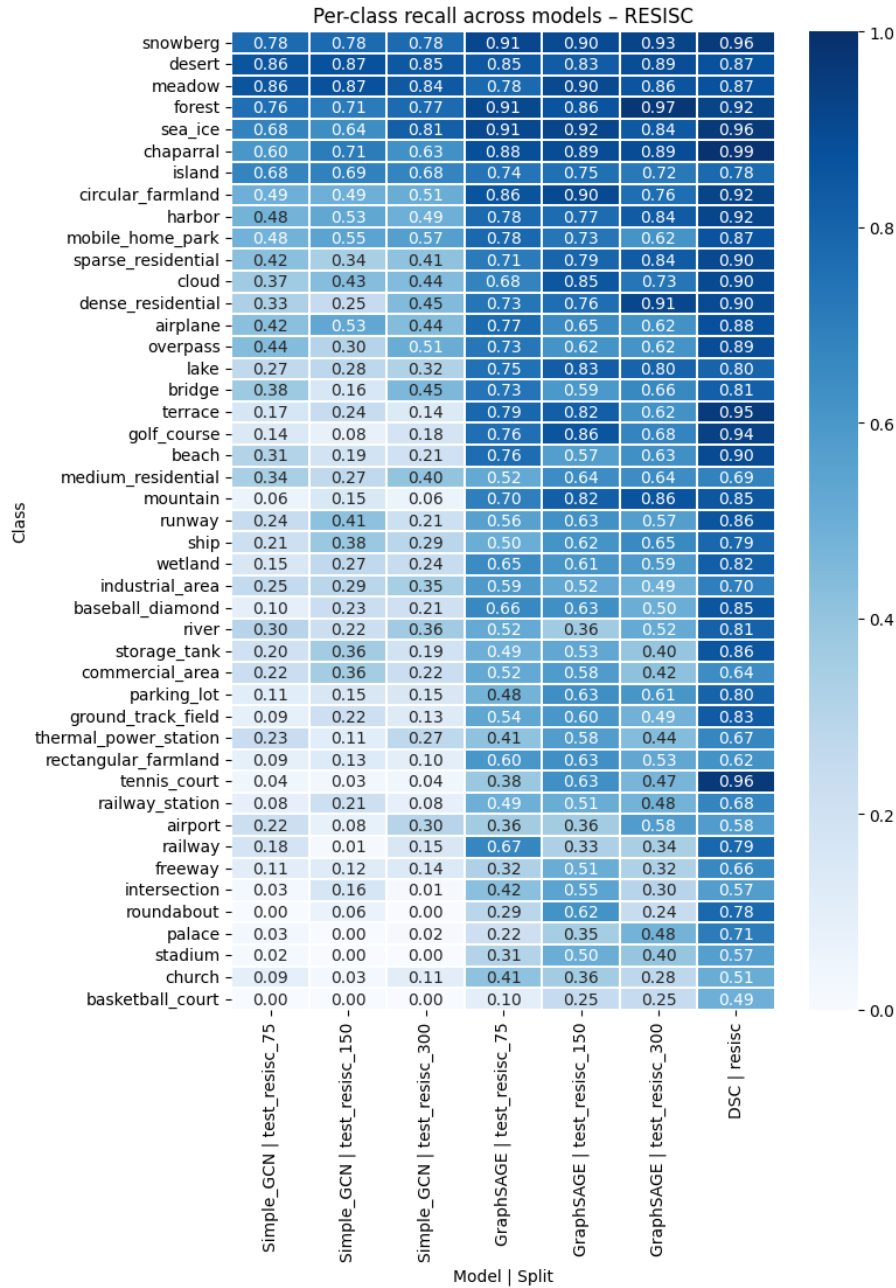
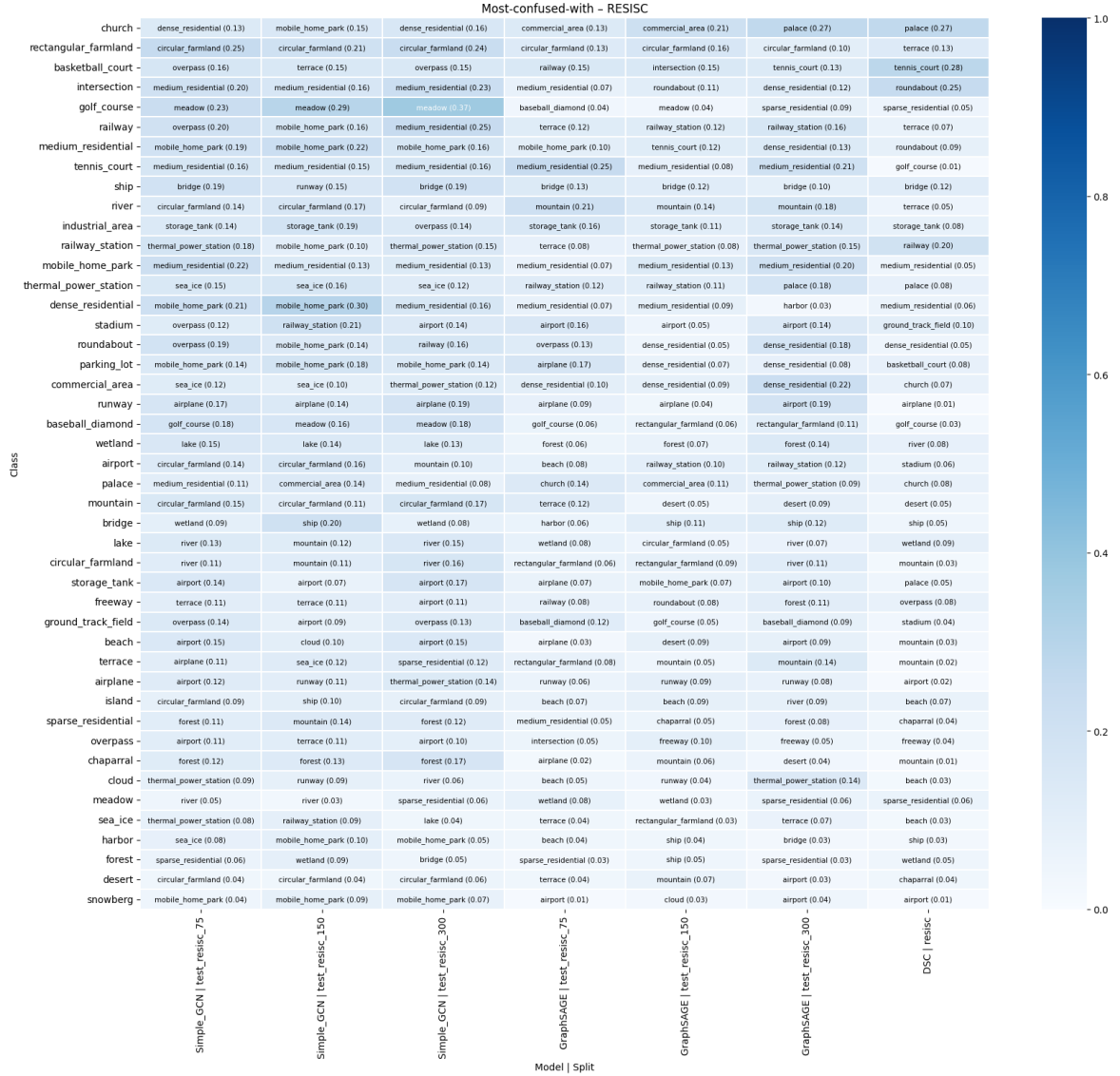


Figure 4.7 shows that the highest confusion that all models have in common appears between classes that naturally fall to similar categories, such as *dense residential-medium residential*, *mobile home park-medium residential*, *industrial area-storage tank* or *rectangular farmland-circular farmland*. The CNN model achieves the lowest overall confusion levels, with its most problematic classes being *basketball court* and *church*.

Figure 4.7 *Most-confused-with classes with the normalized confusion matrix value for RESISC45 (source: Author)*



4.5 Discussion

The conducted experiments demonstrated the differences between CNN and GNN models in both predictive performance and efficiency. They provided the following answers for the research questions set in chapter 3.1.

1. How do GNNs compare in accuracy to CNNs when applied to an image classification task and how do the results differ for different types of datasets?

In terms of accuracy, CNNs outperformed GNNs on both datasets. This result was expected as the CNN architecture is optimized for image grids. The poorer results of the GNN models can be partly credited to the loss of detail that happens during the transformation of images to superpixel-graphs as during this process the image is greatly simplified and condensed to only a few nodes. Despite this loss, the graph

representation still captures some spatial and color relationships and the GNN models, specifically GrapSAGE, managed to get reasonable results. For CIFAR-10, a low resolution dataset, the difference in accuracy between the CNN model and the best performing GraphSAGE model was 0.235 while for RESISC45, a high-resolution dataset the gap was narrowed down to 0.154 which can indicate that the graph conversion removes too much detail in the low resolution dataset.

2. How do convolutional and graph-based architectures compare in terms of the trade-off between test accuracy and computational cost?

Even though CNNs remain superior in terms of accuracy, GNNs offer advantages in computational efficiency. In the performed experiments the CNN models were heavier in parameters, with about 5x more parameters in the best performing model for CIFAR-10 and 65x more for RESISC45. For CIFAR-10 the CNN model showed better accuracy-cost efficiency (for cost considered as both inference time and training time) than the best performing GNN model. This result does not match with the initial expectations of GNNs being more efficient for both datasets, but showed, that for a simple dataset the CNN architecture is both accurate and fast in its predictions and training. For RESISC45 the GNN model showed a much better accuracy-cost efficiency indicating that in resource-constrained scenarios, the GNN approach may be suitable for more complex datasets.

However, it has to be noted that the conversion time to superpixel-graphs was excluded from the efficiency computations even though it is relevant in practice. Since for the more complex RESISC45 dataset this conversion took around 3 hours, regardless the number of segments, adding this time would significantly change the results.

3. How does the number of segments in the image-to-graph conversion method affect the metrics of GNNs?

During the superpixel-graph creation, the number of segments used in the SLIC algorithm plays a role in the preprocessing time as well as the results of accuracy. The GCN model showed slight improvements in accuracy with the increasing number of segments but the GraphSAGE model showed almost no improvements for the CIFAR-10 dataset and even a drop in accuracy for the RESISC45 dataset when increasing the number of segments from 150 to 300. The lower values of accuracy for 75 segments are probably caused by the lack of detail while the drop in accuracy could potentially be caused by over-segmentation. It is therefore important to find the optimal segmentation, in the best case, one that provides a balance between graph-conversion time and accuracy.

4. Do convolutional architectures or graph architectures work better for certain classes of selected datasets?

The class-level analysis showed that often the GNN and CNN models struggle the most with the same classes, such as the *cat-dog* or *truck-automobile* pairs in CIFAR-10 or the *industrial area-storage tank* pair in RESISC45. They can also perform well on the same classes, such as correctly classifying *ship* in CIFAR-10 and

snowberg or *desert* in RESISC45. In case of the RESISC45 dataset, GraphSAGE model even performed very close to the CNN model in correctly classifying several natural categories. Overall, the GNNs performed best on images with distinct colors, simpler shapes and texture, while they had problems with images with finer textures and local details, that could have been easily lost during the conversion to graphs. The performance of the CNN models was a lot more consistent across all classes and generally better in comparison to the GNN models.

In general CNNs remain best suited in image classification tasks that require high accuracy. If the images have busy textures and/or depend on local details, then the GNNs struggle with the classification task. It could be attributed to the SLIC algorithm having problems with finding similar regions to create the superpixels and the high loss of detail during conversion to graphs. This simplification of images is however beneficial for the efficiency of the GNN models.

Conclusions

This thesis compared Convolutional Neural Networks (CNNs) and Graph Neural Networks (GNNs) on an image classification task considering GNNs as a generalization of CNNs on graph data. The work included selecting appropriate datasets (CIFAR-10 and RESISC45), converting images to graphs using the superpixel method, and implementing both simple CNN models and GNN models (Graph Convolutional Networks and GraphSAGE). The experiments were designed to assess four research questions addressing accuracy and computational cost of these models, the effect of number of superpixel segments on GNN metrics and analysis of the per-class classification.

In terms of accuracy, CNNs outperformed GNNs on both datasets. The lower accuracy can be attributed to the loss of detail during the conversion of images to superpixel graphs. This was especially true for the low-resolution CIFAR-10 dataset, however, GNNs still achieved reasonable results for the RESISC45 dataset.

The comparison of CNNs and GNNs in the trade-off between accuracy and computational efficiency differed for the two selected datasets. The CNN model was more efficient on the simpler CIFAR-10 dataset, with good accuracy as well as fast training and inference time. GNNs showed better efficiency on the RESISC45 dataset, suggesting that they offer some advantages in a more resource-constrained environments for more complex datasets.

The number of segments used to create superpixel graphs had an impact on the GNN metrics. Three different numbers of segments were tried: 75, 150 and 300. For GCN, the accuracy improved with a higher number of segments, but for GraphSAGE, accuracy declined when increasing the number of segments from 150 to 300 for the RESISC45 dataset. The lower values of accuracy for low number of segments could be attributed to the lack of detail in the graph while the drop in accuracy for the higher number of segments could be caused by over-segmentation.

The class-level analysis showed, that CNNs were much more consistent and generally better in their classification across classes in both datasets. GNNs struggled on images with finer textures and local detail. They yielded the best results on images with distinct colors, simpler shapes and texture. For the RESISC45 dataset, GNNs even showed comparable results to CNNs in several natural categories. The poorer performance of GNNs could be again attributed to the loss of detail during the conversion to graphs.

While this thesis provided insights into the comparison of CNNs and GNNs from various aspects, there are still several limitations. Firstly the CNN models used were relatively simple and the results could have likely been improved with deeper or pretrained models. Automatic hyperparameter tuning was not used, as the hyperparameters were only tuned using trial and error, meaning that the models were probably not optimized to their full potential, which could have further improved their performance. Furthermore, only the superpixel method of image-to-graph conversion using SLIC was applied, and the impact of

number of segments was only tested with three values. The compactness parameter in SLIC was also only set to one fixed value.

Future research could address these limitations by experimenting with more sophisticated CNN architectures, such as pretrained models, and by including hyperparameter tuning to fully explore the potential of both CNNs and GNNs. Additionally, testing other methods for converting images to superpixels such as Quickshift or Felzenszwalb (Nazir et al., 2021) could provide a more comprehensive understanding. Further studies could also extend this work by applying GNNs to other image datasets or by exploring hybrid models that combine the strengths of both CNNs and GNNs.

List of references

- Abiodun, O. I., Jantan, A., Omolara, A. E., Dada, K. V., Mohamed, N. A., & Arshad, H. (2018). State-of-the-art in artificial neural network applications: A survey. *Heliyon*, 4(11), e00938. <https://doi.org/10.1016/j.heliyon.2018.e00938>
- Achanta, R., Shaji, A., Smith, K., Lucchi, A., Fua, P., & Ssstrunk, S. (2012). SLIC Superpixels Compared to State-of-the-Art Superpixel Methods. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 34(11), 2274–2282. <https://doi.org/10.1109/TPAMI.2012.120>
- Aggarwal, C. C. (2018). *Neural Networks and Deep Learning: A Textbook*. Springer International Publishing. <https://doi.org/10.1007/978-3-319-94463-0>
- Bronstein, M. M., Bruna, J., LeCun, Y., Szlam, A., & Vandergheynst, P. (2017). Geometric deep learning: Going beyond Euclidean data. *IEEE Signal Processing Magazine*, 34(4), 18–42. <https://doi.org/10.1109/MSP.2017.2693418>
- Buduma, N., Buduma, N., & Papa, J. (2022). *Fundamentals of Deep Learning: Designing Next-Generation Machine Intelligence Algorithms*. O'Reilly Media. <https://books.google.cz/books?id=2e5vEAAAQBAJ>
- Cheng, G., Han, J., & Lu, X. (2017). Remote Sensing Image Scene Classification: Benchmark and State of the Art. *Proceedings of the IEEE*, 105(10), 1865–1883. <https://doi.org/10.1109/JPROC.2017.2675998>
- Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., Unterthiner, T., Dehghani, M., Minderer, M., Heigold, G., Gelly, S., Uszkoreit, J., & Houlsby, N. (2021). *An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale* (No. arXiv:2010.11929). arXiv. <https://doi.org/10.48550/arXiv.2010.11929>
- Dubey, S. R., Singh, S. K., & Chaudhuri, B. B. (2022). *Activation Functions in Deep Learning: A Comprehensive Survey and Benchmark* (No. arXiv:2109.14545). arXiv. <https://doi.org/10.48550/arXiv.2109.14545>
- Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press.
- Goyal, N., & Steiner, D. (2022). *Graph Neural Networks for Image Classification and Reinforcement Learning using Graph representations* (No. arXiv:2203.03457). arXiv. <https://doi.org/10.48550/arXiv.2203.03457>
- Hamilton, W. L. (2020). *Graph Representation Learning*.
- Hamilton, W. L., Ying, R., & Leskovec, J. (2018). *Inductive Representation Learning on Large Graphs* (No. arXiv:1706.02216). arXiv. <https://doi.org/10.48550/arXiv.1706.02216>

- Hamilton, W., Ying, Z., & Leskovec, J. (2017). Inductive Representation Learning on Large Graphs. *Advances in Neural Information Processing Systems*, 30. https://proceedings.neurips.cc/paper_files/paper/2017/hash/5dd9db5e033da9c6fb5ba83c7a7e9-Abstract.html
- Kipf, T. N., & Welling, M. (2017). *Semi-Supervised Classification with Graph Convolutional Networks* (No. arXiv:1609.02907). arXiv. <https://doi.org/10.48550/arXiv.1609.02907>
- Krizhevsky, A. (2009). *Learning Multiple Layers of Features from Tiny Images*.
- LeCun, Y., Bengio, Y., & Hinton, G. (2015). Deep learning. *Nature*, 521(7553), 436–444. <https://doi.org/10.1038/nature14539>
- Lecun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11), 2278–2324. <https://doi.org/10.1109/5.726791>
- Ma, Y., & Tang, J. (2021). *Deep Learning on Graphs* (1st edn). Cambridge University Press. <https://doi.org/10.1017/9781108924184>
- Nazir, U., Wang, H., & Taj, M. (2021). *Survey of Image Based Graph Neural Networks* (No. arXiv:2106.06307). arXiv. <https://doi.org/10.48550/arXiv.2106.06307>
- Nikolentzos, G., Thomas, M., Rivera, A. R., & Vazirgiannis, M. (2021). Image Classification Using Graph-Based Representations and Graph Neural Networks. In R. M. Benito, C. Cherifi, H. Cherifi, E. Moro, L. M. Rocha, & M. Sales-Pardo (Eds), *Complex Networks & Their Applications IX* (Vol. 944, pp. 142–153). Springer International Publishing. https://doi.org/10.1007/978-3-030-65351-4_12
- Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Köpf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., ... Chintala, S. (2019). *PyTorch: An Imperative Style, High-Performance Deep Learning Library* (No. arXiv:1912.01703). arXiv. <https://doi.org/10.48550/arXiv.1912.01703>
- Patterson, J., & Gibson, A. (2017). *Deep learning: A practitioner's approach* (First Edition). O'Reilly.
- Rodrigues, J., & Carbonera, J. (2024). Graph Convolutional Networks for Image Classification: Comparing Approaches for Building Graphs from Images: *Proceedings of the 26th International Conference on Enterprise Information Systems*, 437–446. <https://doi.org/10.5220/0012263200003690>
- Sanchez-Lengeling, B., Reif, E., Pearce, A., & Wiltschko, A. B. (2021). A Gentle Introduction to Graph Neural Networks. *Distill*, 6(9), e33. <https://doi.org/10.23915/distill.00033>
- Santurkar, S., Tsipras, D., Ilyas, A., & Madry, A. (2018). How Does Batch Normalization Help Optimization? *Advances in Neural Information Processing Systems*, 31.

<https://proceedings.neurips.cc/paper/2018/hash/905056c1ac1dad141560467e0a99e1cf-Abstract.html>

Tanis, J. H., Giannella, C., & Mariano, A. V. (2024). *Introduction to Graph Neural Networks: A Starting Point for Machine Learning Engineers* (No. arXiv:2412.19419). arXiv. <https://doi.org/10.48550/arXiv.2412.19419>

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł. ukasz, & Polosukhin, I. (2017). Attention is All you Need. *Advances in Neural Information Processing Systems*, 30. https://proceedings.neurips.cc/paper_files/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html

Wu, Z., Pan, S., Chen, F., Long, G., Zhang, C., & Yu, P. S. (2019). *A Comprehensive Survey on Graph Neural Networks*. <https://doi.org/10.48550/ARXIV.1901.00596>

Xu, K., Hu, W., Leskovec, J., & Jegelka, S. (2019). *How Powerful are Graph Neural Networks?* (No. arXiv:1810.00826; Version 3). arXiv. <https://doi.org/10.48550/arXiv.1810.00826>

Xu, X., Zhao, X., Wei, M., & Li, Z. (2023). A comprehensive review of graph convolutional networks: Approaches and applications. *Electronic Research Archive*, 31(7), 4185–4215. <https://doi.org/10.3934/era.2023213>

Yang, F., Sun, Q., Jin, H., & Zhou, Z. (2020). Superpixel Segmentation With Fully Convolutional Networks. *2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 13961–13970. <https://doi.org/10.1109/CVPR42600.2020.01398>

Zaidi, S. S. A., Ansari, M. S., Aslam, A., Kanwal, N., Asghar, M., & Lee, B. (2021). *A Survey of Modern Deep Learning based Object Detection Models* (No. arXiv:2104.11892). arXiv. <https://doi.org/10.48550/arXiv.2104.11892>

Zhao, X., Wang, L., Zhang, Y., Han, X., Deveci, M., & Parmar, M. (2024). A review of convolutional neural networks in computer vision. *Artificial Intelligence Review*, 57(4), 99. <https://doi.org/10.1007/s10462-024-10721-6>

Zhou, J., Cui, G., Hu, S., Zhang, Z., Yang, C., Liu, Z., Wang, L., Li, C., & Sun, M. (2020). Graph neural networks: A review of methods and applications. *AI Open*, 1, 57–81. <https://doi.org/10.1016/j.aiopen.2021.01.001>

Zhou, Y., Huo, H., Hou, Z., & Bu, F. (2023). A deep graph convolutional neural network architecture for graph classification. *PLOS ONE*, 18(3), e0279604. <https://doi.org/10.1371/journal.pone.0279604>

Appendices

Appendix A: Electronic material

- File *Comparison_of_CNNs_and_GNNs.ipynb* contains a python notebook with all code used to conduct the experiments. It consists of preprocessing, conversion of images to graphs, modeling, evaluation and visualizations.
- Folder *superpixel_graphs* contains the converted image datasets to graphs (for the according train, validation and test sets). For each set of the datasets, three versions of the superpixels graphs are saved, corresponding to different segment values (75, 150, 300). Because of the large size (1.42 GB), the folder is saved to Google Drive and can be accessed through this link:
https://drive.google.com/drive/folders/1_J0mYCjIrIF3dQ79LiVI67VzkIDVlITU?usp=sharing
- Folder *models* contains learned parameters of the used models in a .pt format.
- Folder *train_summaries* contains training metrics such as number of parameters, best epoch, total training time etc. for each model.
- Folder *eval* contains files used for evaluation of the models, specifically files *metrics.json* (contains the test accuracy), *timing.json* (contains the inference time), *preds.npy* and *targets.npy* (contain predictions and targets that are used for plotting confusion matrices) for each model.

The source code and the additional folders (except for *superpixel_graphs*) are available as attachments in InSIS.